

HLS Above CppHDL

Design Boundary

HLS is an experimental superstructure over CppHDL, not a change to the RTL contract. Keep HLS-specific source, scheduling, allocation policy, and tests in `hls/`. Reuse the existing Clang AST and Sema specialization machinery so improvements to normal CppHDL parsing remain available to HLS.

Translate actual `std::` container methods. Do not implement substitute containers or select replacement algorithms by container name. For example, synthesizing `std::map` must follow its tree operations, not replace them with a linear key/value table.

The LLVM transaction-kernel experiment has been removed. HLS no longer uses Clang CodeGen, `kernel-source.ll`, `kernel-lowered.ll`, or an external C++ compiler to synthesize a method. Native test executables and Verilator's generated C++ still need a C++ compiler, as usual.

The packet-processing example uses ordinary C++ methods to parse Ethernet/IPv4/UDP/SBE and construct OUCH order frames, with 1,000 randomized quotes checked in native and Verilator flows. Its scheduled FSM can be exported directly to the synthesis graph. Retiming that delayed FSM does not turn it into an II=1 streaming pipeline.

Scheduler Selection

Choose the scheduler in C++, not with a converter scheduling flag:

```
1 cpphdl::hls::ClockedDelayer<ContainerMethods> container;  
2 cpphdl::hls::ClockedPipeline<WordMethods, 3> stream;
```

Both wrappers are declared in `hls/Clocked.h` and use the same command/response ports and `uint64_t command(uint32_t, uint32_t, uint32_t)` entry signature. Ordinary SV conversion and synthesis recognize these wrappers automatically. The wrapper selects the scheduler; `--synth` selects graph export and gate-level synthesis instead of ordinary SV emission. Container diagnostics and bounded recursion outside these wrappers also run automatically. `Clocked<T, ...>` remains a compatibility alias for `ClockedDelayer<T, ...>`.

delayed_logic

`ClockedDelayer` accepts one invocation at a time. Its AST schedule executes straight-line helper calls within a clock, suspending at loops and scheduled memory accesses. Persistent objects, containers, and bounded recursion remain supported. Its native wrapper is a transaction reference, not a cycle-accurate model of variable-duration method execution.

Implementation: `delayed_scheduler.h/.cpp`. Existing scheduler regressions are named `Delayed*.cpp` and registered as `hls_delayed_*`.

External Memory Pointers

Include `hls/Clocked.h` and use `external_memory<T>(byte_address)` to create a pointer into an external controller's address space:

```
1 struct LoadWeight {  
2     uint64_t command(uint32_t base, uint32_t index, uint32_t unused) {  
3         auto weights = cpphdl::hls::external_memory<const uint64_t>(base);  
4         return weights[index];  
5     }  
6 };  
7 cpphdl::hls::ClockedMemory<LoadWeight> loader;
```

`ClockedMemory<T>` uses the delayed scheduler with an explicit external memory channel; it has the same command/response contract as `ClockedDelayer`. Pointer indexing, dereferencing, pointer arithmetic, aliases and helper arguments of type `cpphdl::hls::external_ptr<T>` retain external address-space identity. Local pointers still address local HLS storage. Do not cast external pointers to ordinary pointers. Supported elements are integer types of 1, 2, 4 or 8 bytes, including signed and const-qualified types. Addresses are 32-bit **byte** addresses; each access must be naturally aligned and the caller must keep address arithmetic within that address space. No allocation or model-file loading is implied.

A load suspends the generated method until its response is accepted. A store also suspends until acknowledgement, before subsequent reads or writes execute. The scheduler captures command arguments, saves values needed after the wait, and resumes the dependent statements only after completion. Loops and helper calls can contain accesses. There is one outstanding request per wrapper, no speculation, no bursts, and no implicit cache. `ClockedPipeline` rejects these pointer accesses rather than treating a delayed load as combinational logic.

Connect `loader.memory_out` to a controller through `assignIf`. Its `ExternalMemoryIf` interface has these controller-side members:

- `valid_in, ready_out`: request handshake.
- `write_in, addr_in, size_in, data_in`: operation, byte address, byte count (1/2/4/8), and 64-bit write data.
- `valid_out, ready_in`: completion handshake for **both reads and writes**.
- `data_out, error_out`: 64-bit read data and completion error.

Data occupies the low `size` bytes, little-endian, regardless of address lane. The requester holds the complete request stable until accepted. A controller must return exactly one completion per accepted request and hold its response stable until accepted; it may complete on the request-acceptance edge. A controller with read/write/busy signals needs an adapter to this handshake. AXI4 and DDR PHY/controller implementations are not supplied by this

interface. They must perform their own lane placement, byte strobes, buffering and bus-error handling.

Misaligned accesses set `fault_out=3` without issuing a request. A controller error sets `fault_out=5`; execution stops and a fault response is held until accepted. Reset is required to clear the fault. Reset **both** requester and controller, cancelling outstanding requests and discarding old responses: the channel has no transaction ID to distinguish a late pre-reset completion. The controller is responsible for range/access checks; the pointer does not carry a hardware allocation bound. There is no built-in timeout.

For native functional tests, map a host buffer before running the wrapper:

```
1 cpphdl::hls::bind_external_memory(weights.data(), weights.size()*sizeof(uint64_t),  
2                                0x1000);
```

The native implementation dereferences that buffer and executes a complete command as a transaction. It does **not** simulate DDR timing or drive memory transactions. The binding is thread-local, one contiguous mapping per simulation thread. Pointer creation checks the initial mapped address; ordinary subsequent C++ pointer accesses require the usual valid bounds and object lifetimes. RTL and gate-level Verilator tests in `hls/tests/ExternalPointer.*` exercise the actual waits, stalls, signed/narrow transfers, faults and reset cancellation.

Separating Loading From Computation

Use a delayed loader and a pipelined arithmetic engine as separate child modules. The parent stores loaded operands in two local `memory<>` tiles. It fills one tile while the arithmetic engine consumes the other, releasing a tile only after its last operand has been captured. Ready/valid handshakes prevent buffer overwrite and keep result metadata aligned when retiming changes arithmetic latency. This overlaps memory and compute; it does not guarantee that DDR can keep the pipeline full.

`TiledMatVec.cpp` implements this arrangement: pointer-based weight loading, double-buffered weight tiles, an activation tile, `ClockedPipeline` products, full-width accumulation and a pipelined final adder. See its README for the command protocol and native/RTL/retimed gate tests. The parent is explicit RTL; automatically partitioning an arbitrary transformer into these components is not implemented.

pipelined_logic

`ClockedPipeline<T, STAGES, Argument = uint32_t, Result = uint64_t>` accepts a new invocation every clock when the consumer keeps `response_ready_in` asserted. Several invocations occupy different stages simultaneously. Without stalls, an accepted input produces its output after exactly `STAGES` rising edges, including its acceptance edge. `STAGES` must be between 1 and 64.

The three `command` arguments must have the unsigned native integer type `Argument`, and its return type must be `Result`. Their widths may be 8, 16, 32, 64 or 128 bits; 128-bit native execution requires compiler support for `__uint128_t`. Existing two-parameter instantiations retain the `uint64_t command(uint32_t, uint32_t, uint32_t)` contract. For example:

```
1 cpphdl::hls::ClockedPipeline<Product, 4, uint64_t, __uint128_t> product;
```

The integer LLM prototype uses this form to preserve complete signed Q16.48 products. Its external DDR channel is explicit RTL around the pipeline, not automatic HLS pointer-to-DDR lowering. The full transformer is currently a C++ reference, not a synthesized token processor.

The scheduler translates the method calculations into an acyclic operation graph, distributes dependent operations among stages, and registers operands crossing stage boundaries. Branch predicates, results, and fault values remain aligned. It does not just delay the output of a serialized calculation.

If the output is valid and the consumer is not ready, all stages freeze and `command_ready_out` goes low. Keep the offered command stable until accepted. Clocked reset discards every in-flight invocation. The native wrapper matches this handshake and latency, but evaluates a copy of the C++ object at acceptance and delays its result and state update; generated RTL partitions the calculations.

Floating Feedback

Persistent scalar fields are allowed. Each accepted invocation reads the currently committed object state. Its candidate next state becomes visible when its result enters the last stage, on the same advancing edge. A call accepted on that edge still reads the old state. Bubbles do not update state; backpressure freezes both data and state updates. Reset restores the object's constant initializer and discards all pending updates.

This deliberately does **not** preserve sequential C++ feedback behavior. For example, with three stages, the first three consecutive calls can all read the initial counter. State is a whole-object snapshot, so a later call can also overwrite a field changed by an earlier call. There is no forwarding, interlock, or automatic merging of concurrent updates. An FSM author must design for this delayed reaction, partition independent state, or use `ClockedDelayer` when each call must observe its predecessor's update. Increasing pipeline latency can change decisions, not just delay otherwise identical answers.

Scheduling and Timing Are Separate

`pipeline_scheduler.h/.cpp` chooses initial stages from operation dependency depth, without cell-delay estimates. `STAGES` specifies this initial pipeline. The graph retains the logical transition, state/reset boundaries, ready/valid bindings, and original stage boundaries.

With synthesis `fit_pipeline_retiming`, the synthesis retimer uses estimated cell delays to insert further registers, including inside expanded arithmetic. It aligns branch data, result

metadata, validity, and state updates. The resulting latency is reported in the graph's streaming-region metadata and `timing.json` under `streaming_regions` (`hls_stages`, `latency`, and `initiation_interval`). **II remains one** when the consumer is ready; feedback is not converted to a serialized transaction. Feedback becomes visible after the new latency. The native wrapper models the configured **STAGES**, not a subsequently retimed latency. Verify retimed RTL against a reference using the reported latency.

Current supported source subset:

- Integer calculations, branches, and straight-line helper calls are supported.
- Loops, recursion, static locals, mutable globals, dynamic allocation, escaping pointers, and operations requiring scheduled memory are rejected. There is no fallback to delayed execution.
- Stage placement balances operation dependency depth. It does not promise a technology-specific frequency or split an individual multiply/divide. Function `CPPHDL_ONE_CLOCK` and `CPPHDL_KEEP_BOX` constraints are rejected until pipeline placement can preserve them; they are not silently ignored.
- Pure integer free/static functions marked `CPPHDL_BLACKBOX=module:delay_ns` become opaque combinational calls. Their bodies are not lowered; native C++ still runs the supplied body. Retiming preserves each call and uses its declared delay (zero is permitted). RTL simulation and implementation require separately supplied modules. See the external interface contract.
- State must be trivially copyable and have constant initialization. Arrays requiring scheduled memory and dynamic storage remain unsupported.
- Pipeline retiming rules must include a whole streaming region. Synchronous reset is supported; asynchronous reset for these regions is not yet supported. Unachievable cell/control delays produce an error, not a false timing success.

Implementation: `pipeline_scheduler.h/.cpp`, sharing AST lowering with the delayed scheduler. `Pipeline.cpp` has a fully wired parent and matching C++/Verilator tests at 1, 3, and 7 stages. They check 1,024 consecutive commands, randomized branches and bubbles, long output stalls, tag alignment, and reset with in-flight work. A separate graph test checks that arithmetic spans stages and synthesis cannot serialize it. `PipelineFeedback.cpp` adds mutable 96-bit state, nonzero initialization, conditional updates, bubbles, stalls, and reset. Both examples also run through delay-based retiming and gate-level Verilator testing.

```
1 build/cpphdl --generated-dir build/pipeline hls/tests/Pipeline.cpp -- -Iinclude
2 ctest --test-dir build -R '^hls_pipeline_' --output-on-failure
```

To apply delay-based synthesis retiming, use a new output directory:

```
1 build/cpphdl --synth --top PipelineTop --module PipelineTop \
2 --retiming fit_pipeline_retiming --clock-period-ns 3.205128205 \
3 --output build/pipeline-synth hls/tests/PipelineFeedback.cpp -- -Iinclude
```

The converter detects scheduled wrappers in the selected module hierarchy and exports their hardware graph. The C++ wrapper selects the scheduling policy. The period is the 312 MHz estimate target. Read `timing.json` and validate the resulting latency and feedback behavior before integrating the design.

The HFT example uses `ClockedPipeline` for independent word/header parsing, decisions and explicitly indexed TX words. Its short per-word stream recurrences remain in the RTL wrapper, with explicit registers separating checksum accumulation, validation and sequence filtering. Native and four/eight-stage Verilator regressions check 20,000 uninterrupted input words, packet contents, backpressure and reset. Full-design retiming now estimates 3.15 ns against the 315 MHz target while keeping II=1; the gate regression checks the retimed design with the same packet testbench. This is an estimated timing result, not physical timing closure.

Delayed Class Instantiation

An ordinary class can be a clocked member of an RTL module:

```

1  #include "hls/Clocked.h"
2  #include <vector>
3
4  struct Samples {
5      std::vector<uint32_t> values;
6
7      uint64_t command(uint32_t operation, uint32_t index, uint32_t value) {
8          if (operation == 0) {
9              values.push_back(value);
10             return values.size();
11         }
12         uint64_t sum = 0;
13         for (uint32_t i = 0; i < values.size(); ++i) sum += values[i];
14         return sum;
15     }
16 };
17
18 class Top : public cpphdl::Module {
19 public:
20     cpphdl::hls::ClockedDelayer<Samples> worker;
21     // Expose and connect worker's command/response ports in _assign().
22     void _work(bool reset) { worker._work(reset); }
23     void _strobe() { worker._strobe(); }
24 };

```

`ClockedDelayer<T>` carries the `CPPHDL_HLS_CLOCKED` annotation. It is the explicit opt-in to AST scheduling; `T` itself does not need clock methods, registers, ports, or a `Module` base. See the fully connected parent modules in `DelayedArray.cpp`, `DelayedVector.cpp`, `DelayedMap.cpp`, `DelayedList.cpp`, and `DelayedMultimap.cpp`.

The initial entry contract is deliberately small: `uint64_t command(uint32_t operation, uint32_t index, uint32_t value)`. The wrapper exposes these arguments, `command` valid/ready, `result`, `response` valid/ready, and a fault code. It accepts one outstanding command. Arguments are saved on acceptance. The result remains stable until consumed. Reset cancels the current operation and reconstructs the object before ready is asserted again.

```
1 build/cpphdl --generated-dir build/clocked-vector \
2   hls/tests/std/DelayedVector.cpp -- -Iinclude -stdlib=libc++ -fno-exceptions
3 cmake --build build --target hls_tests
4 ctest --test-dir build -R '^hls_delayed_' --output-on-failure
```

Standard-container examples and their analysis helpers live in `hls/tests/std/`. The Verilator tests generate RTL under `build/hls/tests/std/hls_delayed_<name>-rtl/generated/`, where `<name>` is `Array`, `Vector`, `Map`, or `MapSmall`. Shared-memory test names append `_shared_memory`; `List` and `Multimap` currently use that schedule exclusively. General clocked HLS tests (bindings, recursion, rejection, and code reuse), the shared clocked test harness, and storage-helper tests live in `hls/tests/`.

Source-Based Scheduling

`delayed_scheduler.cpp` follows concrete methods reachable from `T::command()` and the object's initialization. Missing template bodies are instantiated with `Sema`. Calls, fields, offsets, casts, constructors, and control flow come from those declarations, including declarations in the installed standard library. There is no separate C++ compilation or LLVM instruction stream.

Each statically elaborated call has source-named bindings, identified by its method, call site, and bounded recursion depth. In shared-memory mode, eligible recursive calls reuse one body and its locals per depth and bound object, instead of duplicating them for every recursive call path. Scalar parameters, locals, and results become directly assigned logic values. A backwards dataflow analysis adds registers only for values needed after a clock boundary. Independent scalar values become individual typed signals, not slices of a single large packed bit vector. Only values live across clocks are grouped in the saved-state record. `this` binds to the selected source object; reference parameters and local references bind to their targets instead of storing another pointer copy. When evaluating later arguments could change a selected pointer, its value is captured before those arguments, preserving C++ evaluation order.

An actual address use, such as `&local` passed to a pointer-taking function, requires addressable storage. Aggregate objects and container allocations also retain their source layout and addresses. These are source data, not a CPU stack. There are no runtime call pushes, pops, return addresses, or stack pointer. A reused recursive body has a finite caller selector that chooses one of its statically generated continuations; it is not a memory address. The conversion-time scope records only map AST declarations to their statically elaborated values. The generated `phase_reg` selects a source continuation; it does not execute instructions.

The preprocessor builds a graph of source statements and emits named, reusable SV functions and an FSM continuation register. The default schedule is:

- Straight-line operations, branches, and nested calls run in the same clock.
- Entering a loop saves its continuation. Loop iterations are clock steps; after-loop code can run with the final condition check.
- A loop inside a called method suspends the caller too. Its locals and references remain available when execution resumes.
- Nested loops have separate continuations. `break` and `continue` route to the appropriate exit or next iteration, including local cleanup.
- A statically single-pass `do { ... } while (false)` is not a clock boundary. This matters for standard-library assertion macros.

The generated functions form an acyclic call sequence within each clock; they do not recursively call one another. Comments identify instantiated methods and their source locations. The normal converter still handles the containing RTL module, its child instance, and port connections.

Shared Memory Schedule

The fifth `ClockedDelayer` template argument explicitly selects a smaller, slower memory implementation:

```
1 // Real std::map methods; 16-bit addresses, 192-byte allocation pool.
2 // true selects one shared memory port instead of parallel arena accesses.
3 cpphdl::hls::ClockedDelayer<MapSmallMethods, 0, 16, 192, true> worker;
```

The default is `false`; existing loop-only schedules do not change. With `true`, arithmetic, branches, and calls still run together until a loop or an addressable-memory access requires another clock. Direct scalar and known aggregate values do not go through this port. This is resource scheduling of the source methods, not one instruction per clock or a replacement map algorithm.

The port transfers up to eight bytes per clock. Larger accesses use successive transfers, retaining the original address and value. A shared checker validates the entire remaining access before writing any bytes, so an out-of-bounds wide store cannot partially modify memory. Reads return on the following clock; dependent work resumes with that result. Values used after a wait are retained in registers. All three command inputs are captured together at acceptance, even if initializing an addressable method parameter takes another clock.

This backend currently uses the same register-backed arena and logical C++ layout, behind one internal port. It does not yet expose `HlsMemoryIf` or support an externally stalled SRAM. A command remains exclusive until its response; memory waits cannot interleave

two commands. Faults stop further effects until reset. Reset cancels in-flight work and restarts object initialization; it does not clear every byte of the arena.

The `*_shared_memory_native` and `*_shared_memory_verilator` tests exercise Array, Vector, Map, MapSmall, List, Multimap, UnorderedMap, and Reuse with this option. Generated files are under `build/hls/tests/std/hls_delayed_MapSmall_shared_memory-rtl/generated/`, for example.

List exercises linked-node insertion, indexed traversal, updates, erase, clear, and reset. Multimap exercises duplicate keys, counting duplicates, removing one duplicate at a time, sorted traversal, tree removal, clear, and reset. Both flows compare results with actual native standard containers; RTL tests also change command pins while busy and apply response backpressure.

`DelayedUnorderedMap.cpp` uses test-local overrides from `UnorderedMapOverrides.h` for missing library helpers and floating-point load calculations. Its native reference still calls the unmodified standard library. The example keeps the default load factor of 1, limits floating size calculations to nonnegative integers no greater than 2^{24} , and accepts prime-helper inputs through 4096. Violating these helper contracts raises a hardware fault, rather than returning an approximate result. The actual libc++ bucket, node, lookup, insertion, rehash, erase, and clear algorithms are still translated from their source bodies. This is not unrestricted floating-point unordered_map support.

Explicit Function Overrides

When a called function has no source body, or its body is unsuitable for RTL, provide a concrete free function and include `hls/Overrides.h`:

```
1 HLS_OVERRIDE("vendor::calculate")
2 uint32_t calculate_rtl(uint32_t input) {
3     return input * 2;
4 }
```

The original function must still be declared for C++ parsing. The annotation selects the replacement only when lowering calls inside `ClockedDelayer<T>`; it does not alter native C++ calls, redefine a library symbol, or patch library headers. It can replace a missing body or an existing body, including a function whose internals use floating point. Return and parameter types must match. Overloads are selected by their concrete signature, and inline namespaces such as `std::__1` are omitted from the target name. If a named target is used with an unmatched signature, conversion fails rather than silently ignoring the policy. Generated comments identify the original function and the selected replacement.

Replacement functions must have visible bodies and may contain supported loops and calls, which use the normal scheduler. Replacement function templates, nonstatic member targets, and ordinary variadic targets are not supported. Duplicate matches, signature mismatches, and cyclic replacements are rejected. The author is responsible for preserving behavior over

the application's domain; keep the native model as the independent reference and test the domain limits.

`HLS_OVERRIDE_BITS` additionally permits a floating parameter or return value to be represented by an unsigned integer of exactly the same bit width. These integers contain IEEE representation bits, not converted integer values. For example, the test's replacement for `__builtin_cceilf` takes and returns `uint32_t`. Floating values can then be stored and copied, but arithmetic still requires an explicit policy. No SystemVerilog `real` or `shortreal` arithmetic is emitted.

Inline floating operations use reserved override names:

```
1 HLS_OVERRIDE_BITS("@float.*.f32.f32.f32")
2 uint32_t multiply_bits(uint32_t lhs_bits, uint32_t rhs_bits);
3
4 HLS_OVERRIDE_BITS("@float.cast.u64.f32")
5 uint32_t size_to_float_bits(uint64_t size);
```

These are signature examples; each replacement must also supply its body. The name contains the operation, operand types, then result type. Type codes are `f` for floating, `u/i` for unsigned/signed integer, and `b` for Boolean, followed by the width (`b1` for Boolean). An unprovided floating operation is an error. Type-generic Clang builtins can likewise be overridden using their concrete call argument types. None of these policies is enabled globally: the replacement definitions must be present in the design translation unit.

The `unordered_map` policies are deliberately bounded integer implementations, not a general IEEE floating-point library. Native helper tests compare them against `libc++` and native floating point; RTL policy tests check invalid fractions, negative values, NaNs, excessive sizes, unsupported load factors, sticky faults, and recovery after reset.

Generic Clang builtins need explicit lowering: reference casts such as `std::forward`, checked integer arithmetic, address-of, allocation, and byte copy/move/set operations. These are not implementations of container methods. Library assertions and allocation failures produce a fault response.

Real `std::map` Methods

The `map` example stores `std::map<uint32_t, uint32_t>` directly. Its command method uses `operator[]`, `find`, iterators, `erase`, `clear`, and `size`. The generated hardware follows `libc++`'s red-black tree, including node allocation, parent/left/right links, color changes, and rotations. It does not substitute a table scan or another container implementation.

HLS examples use Clang with `libc++`, whose tree algorithm bodies are in headers. No bundled library implementation or `tree.cc` include is needed. Native reference tests and Verilator's C++ testbenches also use `libc++`. The converter itself and non-HLS targets keep their existing standard library. The current regression environment uses `libc++ 21.1.3`. Other library versions may introduce additional AST constructs and need their own validation.

Install Clang, libc++ development headers, and libc++abi. CMake checks that the HLS compiler can compile, link, and run a vector/map program. Select another Clang installation with `-DHLS_CXX=/path/to/clang++`; a nonstandard library directory can be supplied with `-DHLS_LIBCXX_LIBRARY_DIR=/path/to/lib`. The tests pass that compiler's header search paths explicitly to CppHDL so the converter's build-time libstdc++ headers cannot enter the HLS source AST. Projects not building HLS examples can set `-DCPPHDL_BUILD_HLS_TESTS=OFF` without disabling the ordinary CppHDL tests.

```

1 build/cpphdl --generated-dir build/clocked-map \
2   hls/tests/std/DelayedMap.cpp -- -Iinclude -stdlib=libc++ -fno-exceptions
3 cmake --build build --target hls_delayed_Map
4 ctest --test-dir build -R '^hls_delayed_Map_' --output-on-failure

```

Look in `cpphdl_hls_ClockedDelayerMapMethods_R8_A16.sv` for functions whose names retain `__tree_min`, `__tree_next_iter`, `__tree_balance_after_insert`, `__tree_remove`, `__tree_left_rotate`, and `__tree_right_rotate`. Node accesses retain `__left_`, `__right_`, `__parent_`, and `__is_black_`. Generated functions use `hls_<qualified_method>__shared_N`. A call without clock suspension becomes one reusable static function, including its branches and nested same-clock calls. A helper's branch and field read stay together; it does not have separate scheduler functions for copying its return value. These whole-method functions have no `active`, `phase`, or continuation arguments. Identity returns such as `return this` bind directly.

The `Combinational.cpp` pass joins branch arms at their common continuation, propagates same-width scalar copies while their sources remain unchanged, and removes unused copies. It preserves saved values when their source changes, branches disagree, or a nested call can modify reference arguments. It also merges adjacent single-predecessor scheduler blocks without crossing a clock boundary. Methods containing loops still have scheduled continuations. Equivalent method bodies and scheduled blocks share code, with source object addresses and scalar values supplied as arguments. Only scheduled continuations also receive successor states. Call-site and recursion suffixes remain on the caller's storage, not on copies of the function body. Ordinary calls do not add a clock; loop boundaries do. Parameters and locals retain their source names in address constants and individual combinational signals. Unused temporary declarations are omitted; they are not collected into an artificial giant struct. Field accesses use named offset constants containing `__tree_node_base` and `__left_`, rather than unexplained numeric offsets. These names are used by executable RTL, not only comments. Known objects and subobjects use direct packed values and constant field slices, including struct copies, base-class fields, and references to those fields. A method call on a known object passes its identity directly instead of saving `this` in a pointer slot. A reference return is a direct binding when all return paths identify the same known subobject. Pointer parameters can also retain a known target when the method only reads the pointer value (it may still modify the pointee). Only values needed across clock boundaries become registers; the clocked object's persistent state is retained between commands.

Objects with escaping addresses or unresolved dynamic targets still use a byte-addressed register array so references and pointers can alias correctly. This fallback is conservative for

the entire object: it does not yet promote independent fields of an escaping object or infer all possible targets of a mutable pointer. Named address constants do not allocate extra registers. The SV storage uses eight packed byte-lane banks. Logical byte address `a` selects bank `a % 8`, row `a / 8`; source addresses, field offsets, and object sizes do not change. Each bank has its own state value through scheduled blocks and shared functions. Writes select bytes within these banks instead of shifting a mask and a value across the complete arena. Reads assemble and rotate eight-byte windows; unaligned accesses and byte aliases remain supported.

In the default parallel schedule, width-specific helpers do not introduce clocks. A write snapshots its address and value and checks bounds before updating any bank. Subsequent reads in the same clock observe those updates, including when pointers or source and destination overlap. Function signatures omit unused banks, just as they omit other unused state. Banking does not compact native pointer fields or implement SRAM scheduling; those remain separate work.

The sharing pass only parameterizes symbols registered by the scheduler. It compares operations, literal values, argument widths, writable/input modes, symbol alias relationships, branch conditions, and clock boundaries. Different template specializations share code only when their lowered operations match. Fields and constants retain their layout-specific meanings. Unused common state is removed from helper arguments, including from nested calls: a scalar-only helper does not receive the storage array. Arguments use `input`; only mutable values used outside the outlined region return to the caller. Updated values return together in a packed result. The caller snapshots that result before copying values back. This avoids function `inout` arguments, which the tested Yosys Slang frontend rejects even in a minimal example. Generated functions have static lifetime, with no `automatic` declarations or `ref` arguments. They contain no recursive SV calls and execute without suspension. Values that cross a clock boundary live in named module registers, not persistent function locals. Independent calls keep independent saved values, including calls that are suspended inside loops. This same-clock outlining is emitted-code reuse, not clocked hardware sharing or a guarantee of reduced synthesized area. Recursive-body hardware sharing in the shared-memory schedule is a separate optimization, described below.

Scheduled blocks use separate combinational processes, connected by successive versions of the values they change. This preserves same-clock data movement while keeping synthesis process lowering local to each block. These connections are wires, not additional register stages. The default schedule clocks loops; the shared-memory schedule also clocks accesses to the arena and entry into reused recursive bodies.

Header-defined container algorithms are the supported baseline. The tested `std::vector` operations need only the standard headers plus the scheduler's allocation and memory-operation lowering. The libc++ `std::map` example also uses only installed headers. Bounded aligned allocation and C++17 returned-object construction preserve the library's temporary node-owner semantics.

Tree clearing recursively visits subtrees. `ClockedDelayer<MapMethods, 8, 16>` explicitly permits up to eight simultaneously active calls of the same concrete function. The scheduler unfolds these calls into separately named depth-specific values and nonrecursive SV blocks.

A call beyond the bound produces `fault_out == 5`, not a successful partial result. The fault persists until reset. The bound must include the terminal/base-case invocation. It is not a bound on the number of map elements. `ClockedDelayer<T>` defaults to rejecting recursion; explicit bounds may be 1 through 16. Without shared-memory scheduling, branching recursion can still produce excessively large generated code. Different bounds produce distinct module names, such as `_R2` and `_R4`, so both can appear in the same parent. The native transaction reference does not enforce this hardware bound.

With `SHARED_MEMORY=true`, directly recursive functions with scalar parameters and scalar or void results share one implementation per depth and bound object. For example, the two subtree calls in `libc++` tree destruction reuse eight depth-specific bodies at bound eight, rather than expanding into 255 copies. Each invocation captures its arguments before updating those shared inputs and takes a clock to enter the body. Returns select the correct caller, and scalar results are copied out before another invocation can overwrite them. Each depth has separate locals, so a child's call cannot overwrite its parent's live data. Ordinary nonrecursive helpers retain their existing same-clock schedule. Reference/aggregate signatures still use the existing bounded expansion. Different enclosing call-depth contexts stay separate so that calls through another recursive function retain the correct depth limits.

Small Red-Black Map Example

A separate, deliberately small red-black tree example lives in `examples/map`. It uses the same `ClockedDelayer` scheduler, 16-bit addresses, 4 KiB pool, shared register/BRAM backends, and transaction workload as the `std::map` regression. Its insertion, deletion, traversal, and clear are iterative; it is not a replacement for the standard-container tests.

Select the hardware address width

The third template argument selects the address width for a clocked object:

```
1  cpphdl::hls::ClockedDelayer<VectorMethods, 0, 16> vector_worker;
2  cpphdl::hls::ClockedDelayer<MapMethods, 8, 16> map_worker;
```

`ClockedDelayer<T, MAX_RECURSION, ADDRESS_BITS, HEAP_BYTES, SHARED_MEMORY, BLOCK_RAM>` accepts address widths from 8 to 64; the default is 16. All standard-container examples use 16-bit addresses; their complete storage layouts fit within this address range. Explicitly wider addresses remain available for larger arenas. This controls pointer signals, pointer-load/store temporaries, address constants and arithmetic, storage-helper address arguments, and the allocation cursor. It does not narrow integer payloads: `uint64_t` values and command results remain 64 bits. Two instances with different widths generate distinct modules (`_A16`, `_A32`).

Conversion rejects a width that cannot represent the complete storage layout and its end address, including the heap when allocation is used. This option does not resize that heap or select SRAM instead of register-backed storage.

Clang’s source object layout is preserved: field offsets, `sizeof`, and pointer slots in aggregates still follow the parsing target’s ABI. Narrow pointers are zero-extended when stored in those slots and narrowed on read. `size_t`, `ptrdiff_t`, and ordinary integers retain their C++ widths. This avoids changing container layout or truncating payloads merely to reduce address wiring. The native C++ reference continues to use host pointers; the width applies to generated RTL, not the host ABI.

The optional fourth argument sizes the allocation pool in bytes (default 4096). It must be a multiple of 16, from 16 through 16777216. Addressable objects and temporaries occupy additional storage; `MEM_BYTES` in generated RTL reports the total. The native reference still uses the host allocator.

```
1  cpphdl::hls::ClockedDelayer<MapSmallMethods, 0, 16, 192> worker;
```

DelayedMapSmall.cpp uses the real libc++ `std::map` with four entries, arbitrary 32-bit keys and values, insertion, updates, lookup, size, and traversal. It rejects a fifth distinct key before allocating, but permits updates at capacity. It does not erase nodes, so four allocations fit its 192-byte pool between resets. The existing Map example retains its broader insertion, removal, and recursion tests.

Clocked RTL tests use Verilator’s `-fno-inline-funcs` option to retain shared function bodies during model compilation. The functions pass their state explicitly. This does not change the clock schedule or turn off correctness checks. A passing simulation still does not establish useful synthesis area or timing for this experimental backend.

Memory Contract

Two choices must ultimately be explicit:

1. Where container state lives: registers, `memory<>`, or external SRAM.
2. The memory port’s capacity, width, latency, number of ports, and protocol.

Dynamically allocated container objects, such as `std::vector<uint32_t>* values = new std::vector<uint32_t>;`, are not supported as an HLS usage contract. Container examples use direct members; their tests do not include dynamically allocated container-object variants. The working clocked Vector and Map tests do exercise allocation of elements and nodes inside those direct members.

The future intended default is **local/member-created objects use registers; dynamically allocated storage uses SRAM/ports**. A local `std::vector` still dynamically allocates its elements, so object placement alone cannot select the policy for all reachable storage. The container object and its allocated nodes/elements need separate policies.

The default AST prototype maps addressable objects and virtual heap addresses into byte-indexed registers, preserving the Clang object-layout offsets. Shared-memory mode can instead infer block RAM as described below. Neither backend establishes achievable timing

without synthesis and implementation. Only addressable source objects occupy the byte array. Their addresses are fixed at conversion, without reusing storage on function return. Constant objects are limited to 4 KiB; unused constant capacity is not allocated in RTL. The prototype heap defaults to 4096 bytes, with a minimum 16-byte alignment and support for explicit power-of-two alignment up to 4096 bytes. It is monotonic: delete is a no-op, storage is reclaimed on reset. Bounds failures are reported; allocation reuse is not yet implemented. Programs must bound total allocations between resets, not just live size.

`Allocator.h` retains the earlier bounded allocator policy for future AST integration; the clocked vector example uses ordinary `std::allocator`. `Memory.h` and `Sram.cpp` retain the independently tested `HlsMemoryIf` protocol. Their tests do **not** prove that this AST scheduler supports SRAM. The shared-memory schedule above is an internal fixed-latency port. Its default storage is register-backed; the optional sixth template argument selects inferred block RAM:

```
1  cpphdl::hls::ClockedDelayer<MapMethods, 8, 16, 4096, true, false> register_map;
2  cpphdl::hls::ClockedDelayer<MapMethods, 8, 16, 4096, true, true> block_ram_map;
```

`BLOCK_RAM` requires `SHARED_MEMORY`. Both use the same container methods, allocation budget, scheduler, and one-cycle memory-read latency. The RAM backend emits eight independent byte-lane RAM instances with synchronous reads, selected-byte writes, and `ram_style = "block"`. Read alignment is applied after the RAM output registers, so unaligned accesses do not add a clock. RAM contents are not cleared on reset; the scheduled constructor reinitializes the object, as for the register backend. The generated module name includes `_B1` for the RAM variant. Read and write enables are explicitly exclusive, so `no_rw_check` does not change transaction behavior. Each byte lane uses a small generated RAM submodule. Preserve hierarchy through RAM inference and technology mapping (`read_slang --keep-hierarchy` in Yosys), then flatten for logic mapping. This keeps collision analysis local rather than traversing the entire scheduler.

An external SRAM backend still needs to suspend on interface handshakes and use the whole interface. A later arbiter can let several containers share an arena. Per-access clocking is opt-in, never silently substituted for the default.

Tests And Limits

Each working container has its own C++ class and native/Verilator tests. The Verilator flow converts the source, checks for instantiated library methods and loop continuations, and builds the connected parent module. It also verifies that no LLVM `.ll` files were generated.

- Array: indexing, fill, two loop-containing calls in an expression, saved arguments/references, nested for/while/do loops, local destructors, delegating constructors, nonzero-offset downcasts, constant objects (including constructors that distinguish constant evaluation from runtime), out-of-line definitions with renamed parameters, and void return expressions.

- Vector: push/growth/relocation, indexing, fill, erase, clear, and reuse of existing capacity.
- Map: ordered traversal checksums, ascending/descending/shuffled insertion, duplicate updates, missing lookups, size, leaf/two-child deletion, recursive clear, reinsertion, and deterministic mixed operations.
- Containers: changed input pins during execution, one-clock straight-line calls, loop suspension, response backpressure, and reset cancellation/restart.
- Bounded recursion: branching calls with persistent side effects, reuse from multiple call sites, one body per depth, calls through another recursive function, depth-specific values across loops, boundary-depth success, overflow fault/backpressure, and reset recovery.
- Reuse: repeated calls with different arguments, aliased references, saved results across loop-containing calls, 8-bit/32-bit template instances, 16-bit/64-bit locals, and 128-bit aggregate copies. Structural checks require one `mix` body and two width-specific `widen` bodies.
- Bindings: explicit `this`, methods returning `*this`, two independent objects, changing the receiver pointer while evaluating arguments, reference aliasing, address-taken scalar updates, object methods that suspend in loops, inherited methods at nonzero base offsets, aggregate base/member initialization, and `const` receiver methods. Additional cases cover anonymous members and reference initialization, C++17 returned-object elision, named and alternate return paths, destructor timing, and aligned allocation followed by placement construction.
- Address widths: all standard-container examples use 16-bit addresses. Explicit 64-bit instances remain in compatibility tests. The Address test instantiates the same methods at 16 and 32 bits with direct storage, shared register-backed memory, and block RAM. It checks pointer fields, pointers retained across loop clocks, positive/negative pointer differences, and 64-bit payloads. Structural checks verify pointer temporary widths. Conversion rejects invalid widths and oversized layouts, including an oversized object using the default 16-bit width.
- Library selection: native compile/link/run with `libc++`, and converter header selection through both `-stdlib=libc++` and explicit `-nostdinc++` paths. A compile-time guard rejects mixed `libc++/libstdc++` headers.
- Direct values: unaddressed scalar locals must not have byte-memory addresses; only values live across clock boundaries have register assignments. Generated code must not contain saved `this` slots or unresolved source access handles.
- Known aggregates: `DelayedAggregates.cpp` checks copies, source/copy isolation, nested fields, inheritance, aliased references, known pointer arguments, same-target reference returns, partial updates, and loop-carried aggregates. Its generated methods must have no byte-storage accesses. `DelayedBindings` also checks conditional references to different objects across loop clocks.
- Storage helpers: generated read/write functions are tested directly for 8/16/24/32/64/128-bit accesses, little-endian byte order, unaligned addresses, overlapping copies, address snapshotting, bounds, and fault preservation. Invalid writes must leave every byte unchanged. Structural checks forbid raw byte accesses in scheduled method bodies.
- Shared-memory transactions: the same Vector/Map/MapSmall/Reuse commands run

through the single-port schedule, including changed command pins during waits, aliased fields, wide copies, unaligned byte accesses, backpressure and reset. `MemoryPort.cpp` and `MemoryPort.sv` additionally verify rejected wide stores leave every memory bank unchanged, invalid reads do not write memory, faults remain sticky, and reset permits new transactions, for both register and RAM storage.

- Block RAM: all seven standard-container workloads and Reuse run with `BLOCK_RAM=true` against their native C++ reference. Structural checks require unpacked RAM arrays and synchronous reads/writes, with no packed-bank helper accesses. Conversion rejects block RAM without shared-memory scheduling.
- Shared-block unit tests: equivalence under symbol renaming, successor-state parameters, and rejection of unsafe merges across differing widths, modes, aliases, literals, branch conditions, and clock boundaries. Identifier substrings, strings, and comments must not be rewritten as parameters.
- Same-clock methods: the map-style `_S_left/return this` pattern, early returns, nested calls, source mutation after a copy, and reference mutation. Structural checks require whole methods without scheduler arguments and no standalone identity-return function. Graph tests preserve branch joins and clock boundaries.
- Overrides: missing bodies, existing floating-point bodies, overload selection, helper-domain comparisons, forward jumps with scope cleanup, full-expression temporary destruction, and invalid-policy faults with reset recovery.
- Rejection tests: unavailable function bodies without overrides, recursion without a bound, static locals, volatile access, floating-point expressions without policies, indirect calls, and the removed kernel option.

The native wrapper is a transaction reference, not a cycle-accurate HLS simulator. It invokes the original C++ command atomically. Tests compare transaction results against RTL and check RTL timing separately. Generating the same scheduled execution for native C++ is still needed for cycle-accurate integration with other modules.

This is not general STL synthesis yet. Notable limits include SRAM scheduling, allocator reuse, arbitrary entry signatures, concurrent commands, named/CDC clocks, exceptions, virtual/indirect calls, lifetime-extended nontrivial temporaries, bit-field object layouts, and switch/range-for lowering. Forward `goto` to a function-body label preserves exited-scope cleanup; backward jumps and nested target labels are rejected. Use structured loops for back edges. Global constant objects containing mutable fields are rejected. Non-null pointer/reference values in their initializers are also rejected until their address relocations can be represented correctly. Unsupported constructs must produce diagnostics, not placeholder RTL. Loops are not automatically proven terminating and have no hardware watchdog. Keeping a long acyclic call chain in one clock can create a long critical path. Scalar `delete` evaluates its argument once and runs a nonvirtual destructor only for a non-null pointer. Like `std::allocator::deallocate`, it does not yet reclaim the monotonic arena. Array `delete`, virtual destruction through `delete`, and custom deallocation functions are rejected rather than approximated. The flat register representation is also expensive for synthesis optimization. Before known-subobject promotion, a build-only experiment with `libc++` `std::map<uint8_t, uint8_t>` (four entries, 16-bit addresses, a 192-byte allocation pool,

and insert/update/find/size commands) completed Xilinx xc7 mapping using Yosys 0.69+62 with Slang. The flow cleaned dead process outputs early, skipped optional SAT resource sharing, retained operator hierarchy with `techmap -extern`, and used classic ABC for six-input LUT mapping. It produced 7,733,665 LUTs and 4,266 flip-flops, with no unresolved cells and no `check -assert` errors. This is **not practical FPGA area**: repeated wide accesses to the packed byte store remain expensive, and operator boundaries restrict optimization across calls. It is not a lower bound on the hardware needed for a map.

Known-subobject promotion subsequently reduced that experiment’s byte store from 480 to 240 bytes (including the unchanged 192-byte node pool). No local temporary remains allocated in its byte store. At the same pre-technology-map optimization stage, generic cells fell from 5,140 to 3,846 and total wire bits from 2,462,974 to 744,830. These are intermediate Yosys statistics, not LUT counts.

The next experiment compared that promoted-object baseline with eight byte-lane banks. Both complete designs were mapped with Yosys 0.69+62 and Slang to Xilinx xc7 primitives, with DSPs, block RAM, I/O buffers, and clock buffers disabled. Both runs skipped optional SAT resource sharing and used the same classic ABC `strash; if` six-input LUT mapping, without `techmap -extern`:

Resource	Promoted objects, single store	Eight byte-lane banks
LUT1 through LUT6	361,431	259,638
CARRY4	3,059	4,999
Flip-flops	2,282	2,282

Banking reduced LUTs by 28.2%, but increased carry cells. These are complete mapped-design counts, not isolated helper estimates. Both mappings passed `check -assert`. They are not directly comparable with the older 7.7-million-LUT run, which used different operator boundaries.

The byte-key map still takes 193 execution clocks for its 41 test transactions (at most eight per transaction). The larger map regression still takes 2,430 execution clocks for 240 transactions (at most 43). Banking introduces no new clock boundaries. Separate SAT experiments proved the generated 24-, 64-, and 128-bit bank access helpers equivalent to byte-array operations for arbitrary defined input data, addresses, and faults. These proofs cover the helpers, not the complete sequential map.

That parallel result remains far too large for this small map. Banking removes whole-arena shift/mask operations but does not eliminate dynamic row selection or share access hardware across same-clock calls. Logical object layout is unchanged: node pointer fields still occupy host-layout space, and this version retained the allocator’s constant division as hardware. This motivated the following constant-propagation and shared-port experiments. Compact physical pointer fields and typed node storage remain future work.

The next experiments kept that same byte-key map and synthesis flow, but separated pure temporary assignments from fault-guarded effects and added the explicit shared-memory schedule:

Experiment	LUTs	Outcome
Parallel banks, pure calculations without fault-enable muxes	211,959	Still too large
Shared port, bounds checks at each caller	28,269	Repeated checking remained expensive
Shared port with one bounds checker	11,722	Wide initialization still widened the port
Parallel OR request mux	12,687	Rejected; larger than the priority mux
64-bit shared port, including accepted-command registers	8,555	Current shared-memory implementation

The final byte-key design uses 284 CARRY4 cells and 3,482 flip-flops. Its 41 transactions take 735 execution clocks, with at most 66 clocks per transaction, instead of 193 and eight in the parallel schedule. The reduced area is bought with memory-access latency; it is not a same-throughput replacement.

The repository’s `std::map<uint32_t, uint32_t>` MapSmall example, which also includes traversal and bulk updates, maps to 13,318 LUTs, 346 CARRY4 cells, and 3,818 flip-flops with the shared-memory option and the same flow. Its 41 transactions take 858 clocks (at most 62). Both small-map wrappers explicitly reject a fifth distinct key; this limit is in their C++ source, not an inferred limit of `std::map` or a consequence of 16-bit addresses.

The broader Map regression, including erase, clear, and recursive destruction, passes 240 transactions in 11,418 execution clocks (at most 158) in shared-memory mode. See the full-size synthesis measurements below for its mapping outcome.

A separate vector experiment uses the repository’s `std::vector<uint32_t>` workload with an explicit 128-byte allocation pool, rather than its default 4,096-byte pool. It maps to 10,801 LUTs, 631 CARRY4 cells, and 4,778 flip-flops with the same shared-memory schedule and synthesis flow. All 40 transactions pass against the native container in 1,686 execution clocks (at most 125), including growth to eight elements, erase, clear, traversal, and bulk updates. These counts do not describe an arbitrarily large vector.

The simulation checks include backpressure and reset cancellation. None of these mapped designs has undergone placement/routing, timing analysis, or sequential netlist equivalence. Compact pointer storage and better scheduling remain possible improvements. Shorter shared SV functions alone do not imply shared hardware; the explicit memory schedule provides the hardware sharing.

Measuring All Standard Container Examples

After building the HLS tests, generate and test both storage variants, then run:

```

1 ctest --test-dir build -R '^hls_delayed_.*shared_memory(_bram)?_verilator$'
  ↪ --output-on-failure
2 python3 hls/tools/synth_std.py --build build \
3   --yosys build/tools/oss-cad-suite/bin/yosys --memory-mib 6144 \
4   --storage registers --output build/hls/std-synthesis-fast
5 python3 hls/tools/synth_std.py --build build \
6   --yosys build/tools/oss-cad-suite/bin/yosys --memory-mib 6144 \
7   --storage bram --output build/hls/std-synthesis-bram

```

This standalone measurement covers Array, Vector, MapSmall, Map, List, Multimap, and UnorderedMap. It does not resize their pools or edit generated RTL. The analysis-only, expected-rejection tests have no synthesizable design to measure. Use `--containers Map List` to select examples and `--timeout 900` to set the per-example limit in seconds. `--memory-mib` limits process address space (4,096 MiB by default; 0 disables the limit). A resource-limited run has no LUT count, not a count of zero. Yosys must provide the `read_slang` frontend. Use `--append --containers Map --memory-mib 6144` with the same `--output` directory to retry Map while retaining the other measurements.

With the command above, results are written to `build/hls/std-synthesis-fast/README.md` and `results.json` in that directory, with a separate report under `build/hls/std-synthesis-bram/` for block RAM. Each example retains its exact synthesis script, a copy of its input RTL, input hashes, log, and machine-readable cell statistics. The flow targets Xilinx 7-series primitives, disables DSPs, disables BRAM only for `--storage registers`, and uses classic ABC `strash`; if six-input LUT mapping without SAT resource sharing. Unlike the earlier experiment scripts, this runner folds constants before its first unused-wire cleanup to reduce intermediate memory consumption and uses `opt -fast` before technology mapping. The full mux-reduction pass was prohibitively slow for the larger map examples. Results include a flow identifier; `--append` refuses to mix flows or Yosys versions. This flow prioritizes completing the whole comparison over minimum area; its counts must not be mixed with the earlier full-optimization measurements. The table counts LUT1 through LUT6 separately from flip-flops, carry cells, and distributed RAM cells. RAMB18 and RAMB36 cells are counted separately. All successful runs must pass `check -assert`; a BRAM run also fails if no block RAM was actually inferred.

Counts describe the complete test module: container methods, command logic, checks, storage, and scheduler. They are not intrinsic costs of a container type and do not establish timing closure or placed/routed utilization. These lengthy measurements are not added to the normal CTest suite.

These measurements exposed oversized function-result temporaries: shared-memory helpers reserved an arena-sized result even when calling only scalar helpers. The generator now sizes each helper's scratch value for its own callees, and each scheduled block's scratch value for its called helper. This does not change the storage capacity or clock schedule. Structural regressions check the widths; 21 focused C++/Verilator and helper checks pass after the correction.

Current Utilization (2026-09-30)

These results include shared multi-cycle calls and clock-local read reuse. All eight designs use 16-bit addresses and their original allocation pools; no generated RTL was edited. The current converter regenerated all 37 checked clocked RTL variants byte-for-byte identically to the previously tested files. All 16 synthesis runs passed `check -assert` and complete technology mapping.

Yosys is 0.69+62 (0edda7a3a-dirty), with the same `xc7-shared-fast-opt-v2 / xc7-shared-fast-opt-v2-bram-v3` flows described above, a 6,144 MiB address-space limit and 900-second per-design timeout. Total synthesis time was 693.7 seconds without BRAM and 325.8 seconds with BRAM.

Without block RAM (all RAM cell counts are zero):

Test	Arena bytes	LUTs	Flip-flops	CARRY4
Array	96	6,821	2,800	341
Vector	4,192	28,285	36,573	747
MapSmall	240	13,422	3,674	335
Map	4,160	38,508	36,339	597
List	4,144	21,078	34,385	226
Multimap	4,144	41,884	36,385	650
UnorderedMap	4,240	200,441	42,979	42,950
RbMap	4,112	33,342	34,990	592

With block RAM (RAMB18 and distributed-RAM counts are zero):

Test	LUTs	Flip-flops	CARRY4	RAMB36
Array	6,453	2,003	341	8
Vector	11,013	2,977	747	16
MapSmall	12,450	1,693	335	8
Map	22,983	3,014	597	16
List	5,500	1,170	226	16
Multimap	23,444	3,168	650	16
UnorderedMap	181,564	8,998	42,950	16
RbMap	14,580	2,065	592	16

These are complete test-module counts, not equal-capacity container comparisons. Array holds eight `uint32_t` elements; MapSmall explicitly bounds distinct keys to four and uses a 192-byte pool. The other dynamic containers retain their 4,096-byte pools. RbMap is the custom comparison implementation, not `std::map`. Eight separately mapped byte banks still consume substantial physical BRAM; these block counts are not minimum storage requirements.

Compared with the immediately preceding call-sharing BRAM measurements, Map falls from 23,366 to 22,983 LUTs (1.6%), and Multimap from 24,274 to 23,444 (3.4%). RbMap falls from 15,262 to 14,580 BRAM-mode LUTs (4.5%), but its register-backed result **increases**

from 32,158 to 33,342 (3.7%). Fewer reads and execution clocks do not guarantee fewer mapped LUTs. UnorderedMap remains an area problem in both modes. None of these counts establish timing closure.

Exact RTL, hashes, scripts, logs, and statistics are retained in `build/hls/read-reuse-registers/` and `build/hls/read-reuse-bram/`. Reproduce after generating the tested shared-memory variants:

```

1 python3 hls/tools/synth_std.py --yosys build/tools/oss-cad-suite/bin/yosys \
2   --storage registers --memory-mib 6144 \
3   --containers Array Vector MapSmall Map List Multimap UnorderedMap RbMap \
4   --output build/hls/read-reuse-registers
5 python3 hls/tools/synth_std.py --yosys build/tools/oss-cad-suite/bin/yosys \
6   --storage bram --memory-mib 6144 \
7   --containers Array Vector MapSmall Map List Multimap UnorderedMap RbMap \
8   --output build/hls/read-reuse-bram

```

Historical Baseline

The following baseline mappings, before recursive-body hardware reuse, used Yosys 0.69+62 (0edda7a3a-dirty), flow `xc7-shared-fast-opt-v2`, a 6,144 MiB host address-space cap, and a 900-second timeout per example. All seven passed; combined synthesis time was 1,038.7 seconds. No pool was reduced for these measurements.

Baseline Without Block RAM

Test	Address bits	Arena bytes	LUTs	Flip-flops	CARRY4
Array	64	96	8,664	3,394	526
Vector	16	4,192	31,609	36,665	747
MapSmall	16	240	14,948	3,818	367
Map	32	4,160	111,196	46,867	3,262
List	32	4,144	22,116	34,754	321
Multimap	32	4,144	112,347	46,931	3,367
UnorderedMap	32	4,240	209,113	45,585	43,295

All these runs used zero RAM cells. Array contains eight `uint32_t` elements; MapSmall explicitly limits distinct keys to four. The other examples keep their 4,096-byte allocation pools. Arena bytes include addressable object storage as well as the pool, and are not an element-count limit. Different examples expose different operations, so these are workload measurements, not equal-capacity container comparisons. The large Map, Multimap, and UnorderedMap results remain an area problem; successful synthesis alone does not make them efficient.

Baseline With Block RAM

The same workloads, address widths, arena sizes, and recursion bounds were mapped with `BLOCK_RAM=true`. These runs use the same Yosys version and logic mapping options, with flow `xc7-shared-fast-opt-v2-bram-v3`. RAM hierarchy is preserved through memory mapping, then flattened before LUT mapping. All seven passed; combined synthesis time was 850.7 seconds.

Test	LUTs	Flip-flops	CARRY4	RAMB36
Array	8,245	2,597	526	8
Vector	11,642	3,069	747	16
MapSmall	13,981	1,837	367	8
Map	92,945	13,523	3,262	16
List	6,410	1,541	321	16
Multimap	94,404	13,718	3,367	16
UnorderedMap	189,134	11,581	43,295	16

RAMB18 and distributed-RAM counts are zero in these mappings. RAMB36 counts are actual mapped 36-Kibit blocks, not estimates from arena size. The eight byte banks are mapped independently and do not use physical RAM capacity efficiently, especially for the small arenas. These are not minimum BRAM requirements.

At this baseline, register-mode RTL matched the previously measured snapshots apart from source-location comments. All 28 focused native, Verilator, and rejection checks passed, including unaligned/wide accesses, bounds failures without partial writes, backpressure, and reset recovery. All seven workloads have identical command counts and execution-clock totals between the two storage modes.

Block RAM removes much of the storage logic and flip-flops, not the scheduler or container algorithms. Map’s LUT count falls by 16.4%, but remains high; RAM alone does not solve recursive-call expansion and control-logic costs. Neither table establishes placed/routed utilization or timing closure.

Recursive Hardware Reuse

The 92,945-LUT Map result exposed a separate lowering problem: both recursive subtree calls were expanded at every level. Bound eight produced 255 copies of tree destruction, rather than eight depth-specific implementations. The generic shared-memory scheduler now reuses each depth’s body, locals, and caller selector. This does not replace the library’s algorithm or reduce the allocation pool, supported operations, address width, or recursion bound.

With the same BRAM synthesis flow:

Test	LUTs before	LUTs after	Flip-flops after	RAMB36
Map	92,945	35,854	5,878	16
Multimap	94,404	36,556	6,070	16

With the same register-backed synthesis flow (zero RAM cells):

Test	LUTs before	LUTs after	Flip-flops after
Map	111,196	53,740	39,219
Multimap	112,347	54,975	39,283

Map’s 240-command regression changes from 11,418 to 11,435 execution clocks (17 additional clocks, 0.15%); its longest command remains 158 clocks. Multimap’s 67 commands change from 2,833 to 2,834 clocks; its longest command remains 115 clocks. These are workload totals, not a general recursion latency guarantee. The extra clocks come from entering reused recursive bodies.

All 33 focused native, Verilator, helper, and expected-rejection checks pass. They include both shared-memory backends, the unchanged standard-container workloads, structural body/register counts, recursion bounds, repeated calls, cross-function recursion, backpressure, and reset recovery. The snapshots and reports are in `build/hls/std-synthesis-bram-recursion-final/` and `build/hls/std-synthesis-recursion-final/`; their input hashes match the regenerated RTL. These remain synthesis counts, not post-route results.

A separate exclusive-writer OR-network experiment for the memory port increased Map to 38,654 LUTs and was discarded. Shorter or less nested source is not by itself evidence of smaller mapped hardware. The retained Map result is still large and is not a claim of efficient or timing-closed implementation.

16-Bit Pointer Logic

All standard-container examples now select 16-bit addresses, also the default for `ClockedDelayer<T>`. Memory-access temporaries holding pointers use the selected address width instead of the host pointer width. Integer payloads, `size_t`, and `ptrdiff_t` keep their source widths. Stored aggregate layouts are unchanged: pointer slots still occupy their host-ABI size, with zero extension on writes. This change reduces pointer logic, not node sizes or allocation capacity.

Using the same synthesis flows as the recursive-reuse results above:

Test	Storage	LUTs before (32-bit)	LUTs now (16-bit)	Flip-flops now	RAMB36
Map	Registers	53,740	43,104	36,787	0
Multimap	Registers	54,975	46,242	36,803	0
Map	Block RAM	35,854	27,268	3,446	16

Test	Storage	LUTs before (32-bit)	LUTs now (16-bit)	Flip-flops now	RAMB36
Multimap	Block RAM	36,556	27,580	3,588	16

The arenas remain 4,160 and 4,144 bytes, respectively, including their unchanged 4,096-byte pools. Both storage variants retain the same execution-clock totals: Map’s 240 commands take 11,435 clocks (maximum 158 per command); Multimap’s 67 commands take 2,834 clocks (maximum 115). Every mapping passes `check -assert`; the BRAM mappings still use 16 physical RAMB36 cells. These are synthesis-only measurements, not timing-closure results, and the area remains substantial.

Snapshots and reports are in `build/hls/std-synthesis-address16/` and `build/hls/std-synthesis-bram-address16/`. The address-width regression compares 16-bit and 32-bit instances in direct-storage, shared-register-memory, and BRAM flows, including saved pointers and 64-bit payloads. A separate rejection test requires an oversized layout using the default address width to fail conversion rather than wrap its addresses. All 119 HLS CTest cases pass, including native C++17, Verilator, helper, and expected-rejection tests.

Sharing Multi-Cycle Calls

Reusing an SV function’s text does not necessarily reuse its hardware. Separate scheduled callers used to retain separate arguments, locals, and memory-access sequences even when they called the same generated `__shared` helper.

The shared-memory scheduler now gives eligible calls one argument/local set, one schedule, and a caller selector for returning to the appropriate continuation. Eligibility is determined from the AST, not container names: free or static functions returning `void`, with scalar parameters, a pointer parameter, and an indirect write in their body. Calls with known direct-object addresses remain specialized so register accesses do not become arena traffic. This applies to both register-backed and BRAM shared-memory configurations.

Sharing must not introduce combinational feedback between a reused body and its callers. After memory scheduling, the converter removes a provisional call boundary when existing memory/loop boundaries already break every such path. A conditional early-return path can require keeping that call clock. The measured map and multimap workloads below add no clocks.

With unchanged 16-bit addresses, allocation pools, operations, recursion bounds, and the same Xilinx BRAM synthesis flow as above:

Test	LUTs before	LUTs after	Flip-flops before	Flip-flops after	RAMB36
Map	27,268	23,366	3,446	3,062	16
Multimap	27,580	24,274	3,588	3,204	16

Map’s 240 commands still take 11,435 execution clocks, maximum 158 per command. Multimap’s 67 commands still take 2,834 clocks, maximum 115. Map’s mapped rotation-associated register bits fall from 800 to 448; scheduled blocks fall from 723 to 619. Input

snapshots, hashes, and synthesis reports are in `build/hls/map-call-sharing-bram/`. These are synthesis counts, not timing results.

`DelayedSharing.cpp` checks repeated calls, aliased pointer arguments, values that must survive a call, conditional returns without dereferencing a null pointer, specialized local pointers, backpressure, and reset during a shared operation. Structural checks require actual multi-caller schedules in both map variants, not merely shared function text. Scheduler unit tests cover retaining/removing call boundaries without introducing zero-clock cycles. All 133 HLS CTest cases passed, including native C++17, Verilator, and expected-rejection checks. The legacy direct-storage Map test timed out during parallel host compilation, then passed alone in 172 seconds with its original 240-second timeout; no test threshold was changed.

Reusing Reads Within a Clock Region

`hls/MemoryEffects.h/.cpp` tracks available memory values before source accesses are expanded into port transactions. The analysis follows the elaborated call graph, including reads and writes inside helper methods; a C++ `const` method is not assumed to leave pointed-to memory unchanged.

For example, `get(p) + get(p)` needs one load when `get` only reads `*p` and there is no intervening clock or memory modification. Both uses consume the same sample. A dominating load can also serve both sides of an `if` and their continuation. At a join, a sample survives only if it is available on every incoming path. The pass does not speculate a load across a null-pointer guard or hoist separate branch-local reads before the branch.

The analysis tracks pointer copies, access widths, and nonescaping local objects. An unknown/possibly aliased write invalidates arena samples; partial writes invalidate overlapping local values. Direct-storage writes can forward their captured value to a subsequent matching read without another load.

Loop, recursion, and provisional shared-call clock boundaries start with no available memory samples. In port mode, every actual memory transaction also invalidates prior arena knowledge. A completed read supplies a sample for the new zero-time region; a write does not justify forwarding that value across its clock edge. Multi-beat aggregate reads are not retained as reusable samples: their earlier bytes were read on earlier edges. Thus `read(p); read(q); read(p)` still performs three port reads, and a loop still reads memory on each iteration. This is not a cache that assumes exclusive ownership of the arena across clocks.

Value numbering preserves expression operators and widths; it does not prove general pointer arithmetic equivalence or disjointness of dynamic pointers. Unsupported volatile/atomic operations remain rejected. Unknown scheduler effects discard the analysis state conservatively.

Memory-writing helpers are analyzed independently of incoming caller samples and aliases, and their internal samples are not exported to callers. This keeps one reusable outlined body instead of producing a different body and extra sample outputs for each call site. These analysis boundaries add no clocks. Read-only helpers remain transparent to sample reuse.

`DelayedMemoryEffects.cpp` compares native C++17 and generated RTL in all three storage configurations. It covers helper calls, forks/joins, conditional and aliased writes, changed

pointers, partial stores, multi-beat reads, loop boundaries, backpressure, reset, and bounds faults. Exact latency checks require a repeated port read to take two clocks rather than three, while an intervening independent request must prevent reuse. `MemoryEffects_test.cpp` exercises value identities, overlap invalidation, branch intersection, and clock boundaries directly.

All 140 HLS CTest cases passed after targeted reruns. Final regeneration of the other 37 clocked RTL variants matched their tested files byte-for-byte; the large direct-storage Map test passed separately in 177 seconds with its existing timeout. The memory-effect unit test also passed under C++17 UBSan.

Sharing remains conservative: different enclosing call/depth contexts keep separate bodies to preserve recursion bounds. In this map, five call sites per rotation direction become two bodies, one per insertion/deletion context, not one global rotation unit. Nonstatic methods and ordinary read-only helpers are not included in this new optimization. Further sharing needs dependency and re-entry analysis; it must preserve caller values and existing latency.

The earlier direct-container discovery tests for vector, list, map, multimap, and unordered_map remain **analysis-only, expected-rejection tests**. `StdContainers.cpp` records reachable methods and layouts in `hls-analysis.json`; it does not replace their algorithms. These tests also use libc++ headers, but their unwrapped container members do not opt into the `ClockedDelayer<T>` scheduler and are still rejected for RTL emission.

The separate scalar recursion pass in `HLS.cpp` remains available for ordinary CppHDL methods: bounded, numbered functions with an explicit terminal method. It is independent of the clocked-object scheduler's bounded call scheduling; neither mechanism emits recursive SystemVerilog or an unbounded runtime stack.

Synthesis-Only Scheduled Graph Export

Use `cpphdl --synth --top TOP_CLASS --output NEW_DIRECTORY source.cpp` to export the real `ClockedDelayer<T>` state machine or `ClockedPipeline<T>` pipeline to the shared synthesis graph. Scheduler selection is automatic, including wrappers inside child modules, module arrays and base classes. `--synth` is the existing synthesis flag; no separate `--synthesis` is needed. Here `--top` names a C++ module class or a root object variable; `--module` sets the output Verilog module name. Ordinary RTL without wrappers continues to use the native RTL graph lowering; it does not acquire HLS scheduling.

The HLS scheduler operates on C++ ASTs. `synth/ScheduledGraph.cpp` consumes its scheduled blocks before SV function sharing and emits graph operations, registers and memory ports directly. `synth/Mapping.cpp` maps the retimed graph to generic gates. Neither stage invokes Yosys, reparses generated SV, synthesizes the native transaction reference, or uses compiler IR. Container algorithms remain the instantiated source methods. Ordinary conversion emits scheduled SV without exporting a synthesis graph. The former `--hls` flag is no longer accepted; remove it from existing commands.

`--retiming fit_pipeline_retiming --clock-period-ns PERIOD` schedules each FSM edge as a transaction, captures its inputs and commits its writes atomically. The new ad-

mission/commit signals require explicit integration; this is not transparent cycle-preserving retiming. See scheduled HLS graphs for the port contract, artifacts and current restrictions.